

```

%%
=====
% COMPLETE OFDM-MIMO SYSTEM SIMULATION - ALL PLOTS IN ONE FIGURE
% Includes: BER vs SNR, PSD Plots,  $\pi/4$ -QPSK Constellation
% No Toolboxes Required - INDEX ERRORS FIXED
%
=====

clear; clc; close all;
%% ===== PART 1:  $\pi/4$ -QPSK CONSTELLATION =====
fprintf('\n===== PART 1:  $\pi/4$ -QPSK CONSTELLATION DIAGRAM =====\n');
% Generate random data for constellation
num_constellation_symbols = 2000;
test_bits = randi([0 1], 1, num_constellation_symbols * 2);
test_bit_pairs = reshape(test_bits, 2, []);
% Differential  $\pi/4$ -QPSK modulation
constellation_symbols = zeros(1, num_constellation_symbols);
phase = 0;

for i = 1:num_constellation_symbols
    bits = test_bit_pairs(i, :);
    if isequal(bits, [0 0])
        delta_phase = pi/4;
    elseif isequal(bits, [0 1])
        delta_phase = 3*pi/4;
    elseif isequal(bits, [1 1])
        delta_phase = -3*pi/4;
    else % [1 0]
        delta_phase = -pi/4;
    end
    phase = phase + delta_phase;
    constellation_symbols(i) = exp(1j * phase);
end
fprintf('\n $\pi/4$ -QPSK Constellation: 8 phase states, differential encoding\n\n');
%% ===== PART 2: LINE CODING & PSD =====
fprintf('===== PART 2: LINE CODING & POWER SPECTRAL DENSITY =====\n');
% Parameters for line coding
samples_per_bit = 100;
Fs = 10000;
test_bits_psd = [1 0 1 1 0 0 1 0 1 1 1 0 0 1 0 1];
time_axis = (0:length(test_bits_psd)*samples_per_bit-1) / Fs;
% NRZ
nrz_signal = [];
for bit = test_bits_psd
    if bit == 1
        nrz_signal = [nrz_signal ones(1, samples_per_bit)];
    else
        nrz_signal = [nrz_signal -ones(1, samples_per_bit)];
    end
end

% Manchester
manchester_signal = [];
for bit = test_bits_psd
    if bit == 1
        manchester_signal = [manchester_signal ones(1, samples_per_bit/2) -ones(1, samples_per_bit/2)];
    else

```

```

    manchester_signal = [manchester_signal -ones(1, samples_per_bit/2) ones(1, samples_per_bit/2)];
end
end

% AMI
ami_signal = [];
last_polarity = 1;
for bit = test_bits_psd
    if bit == 1
        ami_signal = [ami_signal last_polarity * ones(1, samples_per_bit)];
        last_polarity = -last_polarity;
    else
        ami_signal = [ami_signal zeros(1, samples_per_bit)];
    end
end

% PSD Calculation
window_size = min(256, floor(length(nrz_signal)/4));
noverlap = floor(window_size/2);
nfft = 512;

[psd_nrz, f_nrz] = pwelch(nrz_signal, window_size, noverlap, nfft, Fs, 'centered');
[psd_manchester, f_man] = pwelch(manchester_signal, window_size, noverlap, nfft, Fs, 'centered');
[psd_ami, f_ami] = pwelch(ami_signal, window_size, noverlap, nfft, Fs, 'centered');

fprintf('PSD Analysis Complete: NRZ has DC, Manchester no DC, AMI null at DC\n\n');

%% ===== PART 3: CONVOLUTIONAL ENCODER/DECODER =====
function coded_bits = conv_encode(bits)
    reg = [0 0];
    coded_bits = [];
    for i = 1:length(bits)
        input_bit = bits(i);
        out1 = xor(xor(input_bit, reg(1)), reg(2));
        out2 = xor(input_bit, reg(2));
        coded_bits = [coded_bits out1 out2];
        reg(2) = reg(1);
        reg(1) = input_bit;
    end
end

function decoded_bits = viterbi_decode(recv_bits)
    num_states = 4;
    num_input_bits = floor(length(recv_bits) / 2);

    if num_input_bits < 1
        decoded_bits = [];
        return;
    end

    trellis = [1 3 0 3; 2 4 3 0; 1 3 1 2; 2 4 2 1];

```

```

path_metrics = inf(1, num_states);
path_metrics(1) = 0;
paths = cell(1, num_states);

for t = 1:num_input_bits
    received_pair = recv_bits((t-1)*2 + 1 : t*2);
    new_metrics = inf(1, num_states);
    new_paths = cell(1, num_states);

    for state = 1:num_states
        if isinf(path_metrics(state))
            continue;
        end

        % Input 0
        next0 = trellis(state, 1);
        out0 = [floor(trellis(state,3)/2), mod(trellis(state,3),2)];
        metric0 = path_metrics(state) + sum(out0 ~= received_pair);
        if metric0 < new_metrics(next0)
            new_metrics(next0) = metric0;
            new_paths{next0} = [paths{state}, 0];
        end

        % Input 1
        next1 = trellis(state, 2);
        out1 = [floor(trellis(state,4)/2), mod(trellis(state,4),2)];
        metric1 = path_metrics(state) + sum(out1 ~= received_pair);
        if metric1 < new_metrics(next1)
            new_metrics(next1) = metric1;
            new_paths{next1} = [paths{state}, 1];
        end
    end

    path_metrics = new_metrics;
    paths = new_paths;
end

[~, best_state] = min(path_metrics);
decoded_bits = paths{best_state};
end

```

```

%% ===== PART 4: SIMPLIFIED MIMO CHANNEL =====

```

```

function [rx_signal, H] = simple_mimo_channel(tx_signal, N_tx, N_rx, N_sub, CP_len, noise_var)
    rx_signal = zeros(N_rx, N_sub + CP_len);
    H = zeros(N_rx, N_tx, N_sub);

    % Generate channel matrix
    for sub = 1:N_sub
        for rx = 1:N_rx
            for tx = 1:N_tx
                H(rx, tx, sub) = (randn + 1j*randn) / sqrt(2);
            end
        end
    end

    % Apply channel
    for rx = 1:N_rx
        for tx = 1:N_tx

```

```

        for sub = 1:N_sub
            rx_signal(rx, :) = rx_signal(rx, :) + H(rx, tx, sub) * tx_signal(tx, :);
        end
    end
end

% Add noise
noise = sqrt(noise_var/2) * (randn(size(rx_signal)) + 1j*randn(size(rx_signal)));
rx_signal = rx_signal + noise;
end

%% ===== PART 5: MAIN BER SIMULATION =====
fprintf('===== PART 3: BER vs SNR SIMULATION =====\n');

% System Parameters
N_sub = 16;
CP_len = 4;
N_tx = 2;
N_rx = 2;
SNR_dB = 0:4:20;
num_frames = 10;
bits_per_symbol = 2;

% BER storage
ber_zf = zeros(size(SNR_dB));
ber_mmse = zeros(size(SNR_dB));
ber_no_coding = zeros(size(SNR_dB));
% Store data for constellation plots at specific SNR
constellation_before = [];
constellation_after_zf = [];
constellation_after_mmse = [];
for snr_idx = 1:length(SNR_dB)
    SNR = SNR_dB(snr_idx);
    noise_var = 10^(-SNR/10);

    errors_zf = 0;
    errors_mmse = 0;
    errors_no_coding = 0;
    total_bits_zf = 0;
    total_bits_mmse = 0;
    total_bits_no_code = 0;

    for frame = 1:num_frames
        % Generate data
        bits_per_frame = 160;
        data_bits = randi([0 1], 2, bits_per_frame);

        % Convolutional coding
        coded_bits = zeros(2, bits_per_frame * 2);
        for u = 1:2
            coded_bits(u, :) = conv_encode(data_bits(u, :));
        end

        %  $\pi/4$ -QPSK modulation
        mod_symbols = zeros(2, N_sub);
        for u = 1:2

```

```

bits_needed = N_sub * bits_per_symbol;
if length(coded_bits(u, :)) >= bits_needed
    bits_for_mod = coded_bits(u, 1:bits_needed);
else
    bits_for_mod = [coded_bits(u, :), zeros(1, bits_needed - length(coded_bits(u, :)))];
end

if mod(length(bits_for_mod), 2) ~= 0
    bits_for_mod = [bits_for_mod, 0];
end

bit_pairs = reshape(bits_for_mod, 2, []);

phase = 0;
for sym_idx = 1:min(N_sub, size(bit_pairs, 1))
    bits = bit_pairs(sym_idx, :);
    if isequal(bits, [0 0])
        delta = pi/4;
    elseif isequal(bits, [0 1])
        delta = 3*pi/4;
    elseif isequal(bits, [1 1])
        delta = -3*pi/4;
    else
        delta = -pi/4;
    end
    phase = phase + delta;
    mod_symbols(u, sym_idx) = exp(1j * phase);
end
end

% Store transmitted symbols for constellation comparison
if snr_idx == 3 && frame == 1 % At SNR = 8 dB
    constellation_before = mod_symbols(1, :);
end

% OFDM modulation
ofdm_tx = zeros(2, N_sub + CP_len);
for tx = 1:2
    ifft_input = ifft(mod_symbols(tx, :), N_sub);
    ofdm_tx(tx, :) = [ifft_input(end-CP_len+1:end), ifft_input];
end

% Channel
[rx_signal, H] = simple_mimo_channel(ofdm_tx, N_tx, N_rx, N_sub, CP_len, noise_var);

% OFDM demodulation
rx_symbols = zeros(N_rx, N_sub);
for r = 1:N_rx
    rx_data = rx_signal(r, CP_len+1:end);
    rx_symbols(r, :) = fft(rx_data, N_sub);
end

% Equalization
est_zf = zeros(2, N_sub);
est_mmse = zeros(2, N_sub);

for sym_idx = 1:N_sub
    H_sub = squeeze(H(:, :, sym_idx));

```

```

% ZF
if cond(H_sub) < 1e10
    est_zf(:, sym_idx) = pinv(H_sub) * rx_symbols(:, sym_idx);
else
    est_zf(:, sym_idx) = rx_symbols(1:2, sym_idx);
end

% MMSE
Hh = H_sub';
est_mmse(:, sym_idx) = inv(Hh * H_sub + noise_var * eye(2)) * Hh * rx_symbols(:, sym_idx);
end

% Store equalized symbols for constellation comparison
if snr_idx == 3 && frame == 1
    constellation_after_zf = est_zf(1, :);
    constellation_after_mmse = est_mmse(1, :);
end

% Demodulation for each user
for u = 1:2
    % ZF path
    rx_zf = est_zf(u, :);
    demod_bits_zf = [];
    prev_phase = 0;
    for sym = 1:length(rx_zf)
        curr_phase = angle(rx_zf(sym));
        delta = curr_phase - prev_phase;
        if abs(delta - pi/4) < pi/8
            demod_bits_zf = [demod_bits_zf, 0, 0];
        elseif abs(delta - 3*pi/4) < pi/8
            demod_bits_zf = [demod_bits_zf, 0, 1];
        elseif abs(delta + 3*pi/4) < pi/8
            demod_bits_zf = [demod_bits_zf, 1, 1];
        elseif abs(delta + pi/4) < pi/8
            demod_bits_zf = [demod_bits_zf, 1, 0];
        else
            demod_bits_zf = [demod_bits_zf, 0, 0];
        end
        prev_phase = curr_phase;
    end

    % MMSE path
    rx_mmse = est_mmse(u, :);
    demod_bits_mmse = [];
    prev_phase = 0;
    for sym = 1:length(rx_mmse)
        curr_phase = angle(rx_mmse(sym));
        delta = curr_phase - prev_phase;
        if abs(delta - pi/4) < pi/8
            demod_bits_mmse = [demod_bits_mmse, 0, 0];
        elseif abs(delta - 3*pi/4) < pi/8
            demod_bits_mmse = [demod_bits_mmse, 0, 1];
        elseif abs(delta + 3*pi/4) < pi/8
            demod_bits_mmse = [demod_bits_mmse, 1, 1];
        elseif abs(delta + pi/4) < pi/8
            demod_bits_mmse = [demod_bits_mmse, 1, 0];
        else

```

```

        demod_bits_mmse = [demod_bits_mmse, 0, 0];
    end
    prev_phase = curr_phase;
end

% Viterbi decode
decoded_zf = viterbi_decode(demod_bits_zf);
decoded_mmse = viterbi_decode(demod_bits_mmse);

% Count errors
min_len_zf = min(length(decoded_zf), length(data_bits(u, :)));
if min_len_zf > 0
    errors_zf = errors_zf + sum(decoded_zf(1:min_len_zf) ~= data_bits(u, 1:min_len_zf));
    total_bits_zf = total_bits_zf + min_len_zf;
end

min_len_mmse = min(length(decoded_mmse), length(data_bits(u, :)));
if min_len_mmse > 0
    errors_mmse = errors_mmse + sum(decoded_mmse(1:min_len_mmse) ~= data_bits(u,
1:min_len_mmse));
    total_bits_mmse = total_bits_mmse + min_len_mmse;
end
end

% Uncoded simulation
if frame <= 3
    data_no_code = randi([0 1], 2, 100);
    for u = 1:2
        for sym = 1:50
            bits = data_no_code(u, (sym-1)*2+1:sym*2);
            if isequal(bits, [0 0])
                tx_sym = exp(1j*pi/4);
            elseif isequal(bits, [0 1])
                tx_sym = exp(1j*3*pi/4);
            elseif isequal(bits, [1 1])
                tx_sym = exp(1j*-3*pi/4);
            else
                tx_sym = exp(1j*-pi/4);
            end

            noise_sym = sqrt(noise_var/2) * (randn + 1j*randn);
            rx_sym = tx_sym + noise_sym;
            rx_angle = angle(rx_sym);

            if abs(rx_angle - pi/4) < pi/4
                detected = [0 0];
            elseif abs(rx_angle - 3*pi/4) < pi/4
                detected = [0 1];
            elseif abs(rx_angle + 3*pi/4) < pi/4
                detected = [1 1];
            else
                detected = [1 0];
            end

            errors_no_coding = errors_no_coding + sum(detected ~= bits);
            total_bits_no_code = total_bits_no_code + 2;
        end
    end
end
end

```

```

    end
end

ber_zf(snr_idx) = errors_zf / max(total_bits_zf, 1);
ber_mmse(snr_idx) = errors_mmse / max(total_bits_mmse, 1);
ber_no_coding(snr_idx) = errors_no_coding / max(total_bits_no_code, 1);

fprintf('SNR = %2d dB | ZF BER = %.2e | MMSE BER = %.2e | Uncoded = %.2e\n', ...
    SNR, ber_zf(snr_idx), ber_mmse(snr_idx), ber_no_coding(snr_idx));
end

```

```

%% ===== PART 6: CREATE SEPARATE FIGURES
=====

```

```

% Each figure will open in its own window for easy saving as PNG/JPEG

```

```

%% ===== FIGURE 1:  $\pi/4$ -QPSK Constellation Diagram =====

```

```

figure('Name', 'Figure 1:  $\pi/4$ -QPSK Constellation', 'Position', [100 100 600 500]);

```

```

plot(real(constellation_symbols), imag(constellation_symbols), 'bo', ...
    'MarkerSize', 6, 'MarkerFaceColor', 'b', 'MarkerEdgeColor', 'k');
hold on;

```

```

% Draw ideal phase states (8 phases)

```

```

theta = 0:pi/4:2*pi;
ideal_points = exp(1j * theta);
plot(real(ideal_points), imag(ideal_points), 'ro', 'MarkerSize', 10, ...
    'MarkerFaceColor', 'r', 'MarkerEdgeColor', 'k', 'LineWidth', 2);

```

```

% Draw unit circle

```

```

circle_theta = 0:0.01:2*pi;
plot(cos(circle_theta), sin(circle_theta), 'k--', 'LineWidth', 1);

```

```

grid on; axis equal;
xlabel('In-Phase (I)', 'FontSize', 12, 'FontWeight', 'bold');
ylabel('Quadrature (Q)', 'FontSize', 12, 'FontWeight', 'bold');
title('π/4-QPSK Constellation Diagram', 'FontSize', 14, 'FontWeight', 'bold');
legend('Modulated Symbols', 'Ideal Phase States (8 phases)', 'Unit Circle', ...
    'Location', 'best', 'FontSize', 10);
xlim([-1.5 1.5]); ylim([-1.5 1.5]);

```

```

% Add annotation

```

```

text(-1.4, -1.3, '8 Phase States: 0°, 45°, 90°, 135°', 'FontSize', 9);
text(-1.4, -1.45, '180°, 225°, 270°, 315°', 'FontSize', 9);

```

```

% Save the figure

```

```

saveas(gcf, 'Figure1_Constellation.png');
saveas(gcf, 'Figure1_Constellation', 'epsc');
fprintf('Figure 1 saved: Figure1_Constellation.png\n');

```

```

%% ===== FIGURE 2: NRZ Line Coding Time Domain =====

```



```

figure('Name', 'Figure 2: NRZ Line Coding', 'Position', [100 100 800 400]);

plot(time_axis(1:min(2000,length(time_axis))), nrz_signal(1:min(2000,length(nrz_signal))), 'b',
'LineWidth', 1.5);
grid on;
xlabel('Time (seconds)', 'FontSize', 12, 'FontWeight', 'bold');
ylabel('Amplitude', 'FontSize', 12, 'FontWeight', 'bold');
title('NRZ (Non-Return-to-Zero) Line Coding', 'FontSize', 14, 'FontWeight', 'bold');
ylim([-1.5 1.5]);

saveas(gcf, 'Figure2_NRZ_TimeDomain.png');
fprintf('Figure 2 saved: Figure2_NRZ_TimeDomain.png\n');

%% ===== FIGURE 3: Manchester Line Coding Time Domain =====
figure('Name', 'Figure 3: Manchester Line Coding', 'Position', [100 100 800 400]);

plot(time_axis(1:min(2000,length(time_axis))),
manchester_signal(1:min(2000,length(manchester_signal))), 'r', 'LineWidth', 1.5);
grid on;
xlabel('Time (seconds)', 'FontSize', 12, 'FontWeight', 'bold');
ylabel('Amplitude', 'FontSize', 12, 'FontWeight', 'bold');
title('Manchester Line Coding', 'FontSize', 14, 'FontWeight', 'bold');
ylim([-1.5 1.5]);

saveas(gcf, 'Figure3_Manchester_TimeDomain.png');
fprintf('Figure 3 saved: Figure3_Manchester_TimeDomain.png\n');

%% ===== FIGURE 4: AMI Line Coding Time Domain =====
figure('Name', 'Figure 4: AMI Line Coding', 'Position', [100 100 800 400]);

plot(time_axis(1:min(2000,length(time_axis))), ami_signal(1:min(2000,length(ami_signal))), 'g',
'LineWidth', 1.5);
grid on;
xlabel('Time (seconds)', 'FontSize', 12, 'FontWeight', 'bold');
ylabel('Amplitude', 'FontSize', 12, 'FontWeight', 'bold');
title('AMI (Alternate Mark Inversion) Line Coding', 'FontSize', 14, 'FontWeight', 'bold');
ylim([-1.5 1.5]);

saveas(gcf, 'Figure4_AMI_TimeDomain.png');
fprintf('Figure 4 saved: Figure4_AMI_TimeDomain.png\n');

%% ===== FIGURE 5: Power Spectral Density (PSD) Comparison =====
figure('Name', 'Figure 5: Power Spectral Density Comparison', 'Position', [100 100 800 600]);

plot(f_nrz/1000, 10*log10(psd_nrz + eps), 'b', 'LineWidth', 2); hold on;
plot(f_man/1000, 10*log10(psd_manchester + eps), 'r', 'LineWidth', 2);
plot(f_ami/1000, 10*log10(psd_ami + eps), 'g', 'LineWidth', 2);
grid on;
xlabel('Frequency (kHz)', 'FontSize', 12, 'FontWeight', 'bold');
ylabel('Power Spectral Density (dB/Hz)', 'FontSize', 12, 'FontWeight', 'bold');

```

```

title('Power Spectral Density Comparison', 'FontSize', 14, 'FontWeight', 'bold');
legend('NRZ', 'Manchester', 'AMI', 'Location', 'best', 'FontSize', 11);
xlim([-5 5]);

saveas(gcf, 'Figure5_PSD_Comparison.png');
fprintf('Figure 5 saved: Figure5_PSD_Comparison.png\n');

%% ===== FIGURE 6: BER vs SNR Performance =====
figure('Name', 'Figure 6: BER vs SNR Performance', 'Position', [100 100 800 600]);

semilogy(SNR_dB, ber_zf, 'b-o', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'b'); hold on;
semilogy(SNR_dB, ber_mmse, 'r-s', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'r');
semilogy(SNR_dB, ber_no_coding, 'k-^', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'k');

% Add theoretical curve
EbN0_linear = 10.^(SNR_dB/10);
ber_theory = 0.5 * erfc(sqrt(EbN0_linear));
semilogy(SNR_dB, ber_theory, 'g--', 'LineWidth', 2);

grid on;
xlabel('SNR (dB)', 'FontSize', 12, 'FontWeight', 'bold');
ylabel('Bit Error Rate (BER)', 'FontSize', 12, 'FontWeight', 'bold');
title('BER Performance: ZF vs MMSE Equalization', 'FontSize', 14, 'FontWeight', 'bold');
legend('ZF + Convolutional Coding', 'MMSE + Convolutional Coding', ...
    'Uncoded  $\pi/4$ -QPSK', 'Theoretical BPSK', 'Location', 'southwest', 'FontSize', 11);
xlim([SNR_dB(1) SNR_dB(end)]);
ylim([1e-5 1]);

saveas(gcf, 'Figure6_BER_vs_SNR.png');
fprintf('Figure 6 saved: Figure6_BER_vs_SNR.png\n');

%% ===== FIGURE 7: ZF Equalizer Performance Only =====
figure('Name', 'Figure 7: ZF Equalizer Performance', 'Position', [100 100 600 500]);

semilogy(SNR_dB, ber_zf, 'b-o', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'b');
grid on;
xlabel('SNR (dB)', 'FontSize', 12, 'FontWeight', 'bold');
ylabel('Bit Error Rate (BER)', 'FontSize', 12, 'FontWeight', 'bold');
title('ZF Equalizer Performance with Convolutional Coding', 'FontSize', 14, 'FontWeight', 'bold');
xlim([SNR_dB(1) SNR_dB(end)]);
ylim([1e-5 1]);

saveas(gcf, 'Figure7_ZF_Equalizer.png');
fprintf('Figure 7 saved: Figure7_ZF_Equalizer.png\n');

%% ===== FIGURE 8: MMSE Equalizer Performance Only =====
figure('Name', 'Figure 8: MMSE Equalizer Performance', 'Position', [100 100 600 500]);

semilogy(SNR_dB, ber_mmse, 'r-s', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'r');

```

```

grid on;
xlabel('SNR (dB)', 'FontSize', 12, 'FontWeight', 'bold');
ylabel('Bit Error Rate (BER)', 'FontSize', 12, 'FontWeight', 'bold');
title('MMSE Equalizer Performance with Convolutional Coding', 'FontSize', 14, 'FontWeight', 'bold');
xlim([SNR_dB(1) SNR_dB(end)]);
ylim([1e-5 1]);

saveas(gcf, 'Figure8_MMSE_Equalizer.png');
fprintf('Figure 8 saved: Figure8_MMSE_Equalizer.png\n');

%% ===== FIGURE 9: ZF vs MMSE Direct Comparison =====
figure('Name', 'Figure 9: ZF vs MMSE Comparison', 'Position', [100 100 700 500]);

semilogy(SNR_dB, ber_zf, 'b-o', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'b'); hold on;
semilogy(SNR_dB, ber_mmse, 'r-s', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'r');
grid on;
xlabel('SNR (dB)', 'FontSize', 12, 'FontWeight', 'bold');
ylabel('Bit Error Rate (BER)', 'FontSize', 12, 'FontWeight', 'bold');
title('ZF vs MMSE Equalizer Comparison', 'FontSize', 14, 'FontWeight', 'bold');
legend('ZF Equalizer', 'MMSE Equalizer', 'Location', 'southwest', 'FontSize', 12);
xlim([SNR_dB(1) SNR_dB(end)]);
ylim([1e-5 1]);

saveas(gcf, 'Figure9_ZF_vs_MMSE.png');
fprintf('Figure 9 saved: Figure9_ZF_vs_MMSE.png\n');

%% ===== FIGURE 10: Coding Gain Analysis =====
figure('Name', 'Figure 10: Coding Gain Analysis', 'Position', [100 100 800 500]);

subplot(1,2,1);
semilogy(SNR_dB, ber_no_coding, 'k-^', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'k'); hold on;
semilogy(SNR_dB, ber_mmse, 'r-s', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'r');
grid on;
xlabel('SNR (dB)', 'FontSize', 11, 'FontWeight', 'bold');
ylabel('BER', 'FontSize', 11, 'FontWeight', 'bold');
title('Uncoded vs Coded Performance', 'FontSize', 12, 'FontWeight', 'bold');
legend('Uncoded  $\pi/4$ -QPSK', 'Rate 1/2 Convolutional Code + MMSE', 'Location', 'southwest', 'FontSize', 9);
xlim([SNR_dB(1) SNR_dB(end)]);
ylim([1e-5 1]);

subplot(1,2,2);
coding_gain_db = 10*log10(ber_no_coding ./ ber_mmse);
bar(SNR_dB, coding_gain_db, 'FaceColor', [0.2 0.6 0.8]);
grid on;
xlabel('SNR (dB)', 'FontSize', 11, 'FontWeight', 'bold');
ylabel('Coding Gain (dB)', 'FontSize', 11, 'FontWeight', 'bold');
title('Coding Gain vs SNR', 'FontSize', 12, 'FontWeight', 'bold');
for i = 1:length(SNR_dB)
    if ~isnan(coding_gain_db(i)) && coding_gain_db(i) < 10
        text(SNR_dB(i), coding_gain_db(i)+0.3, sprintf('%0.1f', coding_gain_db(i)), ...

```

```

        'HorizontalAlignment', 'center', 'FontSize', 9);
    end
end

sgtitle('Convolutional Coding Gain Analysis (Rate 1/2, K=3)', 'FontSize', 14, 'FontWeight', 'bold',
'Color', 'blue');

saveas(gcf, 'Figure10_Coding_Gain.png');
fprintf('Figure 10 saved: Figure10_Coding_Gain.png\n');

%% ===== FIGURE 11: Line Coding Combined Time Domain =====
figure('Name', 'Figure 11: All Line Coding Waveforms', 'Position', [100 100 800 700]);

subplot(3,1,1);
plot(time_axis(1:min(1500,length(time_axis))), nrz_signal(1:min(1500,length(nrz_signal))), 'b',
'LineWidth', 1.5);
title('(a) NRZ Line Coding', 'FontSize', 11, 'FontWeight', 'bold');
ylabel('Amplitude'); grid on; ylim([-1.5 1.5]);

subplot(3,1,2);
plot(time_axis(1:min(1500,length(time_axis))),
manchester_signal(1:min(1500,length(manchester_signal))), 'r', 'LineWidth', 1.5);
title('(b) Manchester Line Coding', 'FontSize', 11, 'FontWeight', 'bold');
ylabel('Amplitude'); grid on; ylim([-1.5 1.5]);

subplot(3,1,3);
plot(time_axis(1:min(1500,length(time_axis))), ami_signal(1:min(1500,length(ami_signal))), 'g',
'LineWidth', 1.5);
title('(c) AMI Line Coding', 'FontSize', 11, 'FontWeight', 'bold');
xlabel('Time (seconds)', 'FontSize', 11, 'FontWeight', 'bold');
ylabel('Amplitude'); grid on; ylim([-1.5 1.5]);

sgtitle('Line Coding Techniques - Time Domain Comparison', 'FontSize', 14, 'FontWeight', 'bold',
'Color', 'blue');

saveas(gcf, 'Figure11_All_LineCoding.png');
fprintf('Figure 11 saved: Figure11_All_LineCoding.png\n');

%% ===== FIGURE 12: Channel Frequency Response =====
if exist('H', 'var')
    figure('Name', 'Figure 12: Channel Frequency Response', 'Position', [100 100 800 500]);

    subplot(1,2,1);
    freq_axis = 1:size(H, 3);
    channel_mag = abs(squeeze(H(1,1,:)));
    plot(freq_axis, 20*log10(channel_mag + eps), 'b', 'LineWidth', 1.5);
    grid on;
    xlabel('Subcarrier Index', 'FontSize', 11, 'FontWeight', 'bold');
    ylabel('Magnitude (dB)', 'FontSize', 11, 'FontWeight', 'bold');
    title('Channel Frequency Response (H_{11})', 'FontSize', 12, 'FontWeight', 'bold');

```

```

subplot(1,2,2);
channel_phase = angle(squeeze(H(1,1,:))) * 180/pi;
plot(freq_axis, channel_phase, 'r', 'LineWidth', 1.5);
grid on;
xlabel('Subcarrier Index', 'FontSize', 11, 'FontWeight', 'bold');
ylabel('Phase (degrees)', 'FontSize', 11, 'FontWeight', 'bold');
title('Channel Phase Response ( $\angle H_{11}$ )', 'FontSize', 12, 'FontWeight', 'bold');

sgtitle('MIMO Channel Frequency Response Analysis', 'FontSize', 14, 'FontWeight', 'bold', 'Color',
'blue');

saveas(gcf, 'Figure12_Channel_Response.png');
fprintf('Figure 12 saved: Figure12_Channel_Response.png\n');
end

%% ===== FIGURE 13: Constellation Before/After Equalization =====
if ~isempty(constellation_before) && ~isempty(constellation_after_zf)
figure('Name', 'Figure 13: Constellation Comparison', 'Position', [100 100 900 700]);

subplot(2,2,1);
plot(real(constellation_before), imag(constellation_before), 'b.', 'MarkerSize', 8);
grid on; axis equal;
xlabel('In-Phase (I)', 'FontSize', 10); ylabel('Quadrature (Q)', 'FontSize', 10);
title('(a) Transmitted Constellation ( $\pi/4$ -QPSK)', 'FontSize', 11, 'FontWeight', 'bold');
xlim([-1.5 1.5]); ylim([-1.5 1.5]);

subplot(2,2,2);
plot(real(constellation_after_zf), imag(constellation_after_zf), 'r.', 'MarkerSize', 8);
grid on; axis equal;
xlabel('In-Phase (I)', 'FontSize', 10); ylabel('Quadrature (Q)', 'FontSize', 10);
title('(b) Received After ZF Equalization', 'FontSize', 11, 'FontWeight', 'bold');
xlim([-1.5 1.5]); ylim([-1.5 1.5]);

subplot(2,2,3);
plot(real(constellation_after_mmse), imag(constellation_after_mmse), 'g.', 'MarkerSize', 8);
grid on; axis equal;
xlabel('In-Phase (I)', 'FontSize', 10); ylabel('Quadrature (Q)', 'FontSize', 10);
title('(c) Received After MMSE Equalization', 'FontSize', 11, 'FontWeight', 'bold');
xlim([-1.5 1.5]); ylim([-1.5 1.5]);

subplot(2,2,4);
plot(real(constellation_after_zf), imag(constellation_after_zf), 'r.', 'MarkerSize', 6); hold on;
plot(real(constellation_after_mmse), imag(constellation_after_mmse), 'g.', 'MarkerSize', 6);
grid on; axis equal;
xlabel('In-Phase (I)', 'FontSize', 10); ylabel('Quadrature (Q)', 'FontSize', 10);
title('(d) ZF (red) vs MMSE (green) Comparison', 'FontSize', 11, 'FontWeight', 'bold');
legend('ZF', 'MMSE', 'Location', 'best', 'FontSize', 9);
xlim([-1.5 1.5]); ylim([-1.5 1.5]);

sgtitle('Constellation Analysis at SNR = 8 dB', 'FontSize', 14, 'FontWeight', 'bold', 'Color', 'blue');

saveas(gcf, 'Figure13_Constellation_Comparison.png');
fprintf('Figure 13 saved: Figure13_Constellation_Comparison.png\n');
end

```

%% ===== FIGURE 14: Spectral Efficiency Comparison =====

```
figure('Name', 'Figure 14: Spectral Efficiency Comparison', 'Position', [100 100 600 500]);
```

```
ma_schemes = {'FDMA', 'TDMA', 'CDMA'};
```

```
spectral_eff_fdma = 1/2.5; % 1 Mbps / 2.5 MHz = 0.4 bps/Hz
```

```
spectral_eff_tdma = 1/5; % 1 Mbps / 5 MHz = 0.2 bps/Hz
```

```
spectral_eff_cdma = (1*16)/5; % (1 Mbps * 16) / 5 MHz = 3.2 bps/Hz
```

```
spectral_eff_values = [spectral_eff_fdma, spectral_eff_tdma, spectral_eff_cdma];
```

```
bar(spectral_eff_values, 'FaceColor', [0.2 0.6 0.8], 'EdgeColor', 'k', 'LineWidth', 1.5);
```

```
set(gca, 'XTickLabel', ma_schemes, 'FontSize', 12);
```

```
xlabel('Multiple Access Scheme', 'FontSize', 12, 'FontWeight', 'bold');
```

```
ylabel('Spectral Efficiency (bps/Hz)', 'FontSize', 12, 'FontWeight', 'bold');
```

```
title('Spectral Efficiency Comparison for 2 Users', 'FontSize', 14, 'FontWeight', 'bold');
```

```
grid on;
```

```
for i = 1:3
```

```
    text(i, spectral_eff_values(i)+0.1, sprintf('%.1f', spectral_eff_values(i)), ...
```

```
        'HorizontalAlignment', 'center', 'FontSize', 12, 'FontWeight', 'bold');
```

```
end
```

```
ylim([0 4]);
```

```
saveas(gcf, 'Figure14_Spectral_Efficiency.png');
```

```
fprintf('Figure 14 saved: Figure14_Spectral_Efficiency.png\n');
```

%% ===== FIGURE 15: Multiple Access Interference Comparison =====

```
figure('Name', 'Figure 15: Multiple Access Comparison', 'Position', [100 100 800 500]);
```

```
% Create table-like visualization
```

```
subplot(1,3,1);
```

```
text(0.5, 0.8, 'FDMA', 'FontSize', 14, 'FontWeight', 'bold', 'HorizontalAlignment', 'center');
```

```
text(0.5, 0.6, 'Bandwidth/User: 2.5 MHz', 'FontSize', 10, 'HorizontalAlignment', 'center');
```

```
text(0.5, 0.5, 'Interference: Adjacent Channel', 'FontSize', 10, 'HorizontalAlignment', 'center');
```

```
text(0.5, 0.4, 'Complexity: Low', 'FontSize', 10, 'HorizontalAlignment', 'center');
```

```
text(0.5, 0.3, 'Pros: Simple, No MAI', 'FontSize', 10, 'HorizontalAlignment', 'center');
```

```
text(0.5, 0.2, 'Cons: Low Spectral Eff', 'FontSize', 10, 'HorizontalAlignment', 'center');
```

```
axis off;
```

```
title('FDMA', 'FontSize', 12, 'FontWeight', 'bold');
```

```
subplot(1,3,2);
```

```
text(0.5, 0.8, 'TDMA', 'FontSize', 14, 'FontWeight', 'bold', 'HorizontalAlignment', 'center');
```

```
text(0.5, 0.6, 'Bandwidth/User: 5 MHz (shared)', 'FontSize', 10, 'HorizontalAlignment', 'center');
```

```
text(0.5, 0.5, 'Interference: None (perfect sync)', 'FontSize', 10, 'HorizontalAlignment', 'center');
```

```
text(0.5, 0.4, 'Complexity: Medium', 'FontSize', 10, 'HorizontalAlignment', 'center');
```

```
text(0.5, 0.3, 'Pros: No Interference', 'FontSize', 10, 'HorizontalAlignment', 'center');
```

```
text(0.5, 0.2, 'Cons: Critical Sync Required', 'FontSize', 10, 'HorizontalAlignment', 'center');
```

```
axis off;
```

```
title('TDMA', 'FontSize', 12, 'FontWeight', 'bold');
```

```
subplot(1,3,3);
```

```
text(0.5, 0.8, 'CDMA', 'FontSize', 14, 'FontWeight', 'bold', 'HorizontalAlignment', 'center');
```

```

text(0.5, 0.6, 'Bandwidth/User: 312.5 kHz', 'FontSize', 10, 'HorizontalAlignment', 'center');
text(0.5, 0.5, 'Interference: MAI (Near-Far)', 'FontSize', 10, 'HorizontalAlignment', 'center');
text(0.5, 0.4, 'Complexity: High', 'FontSize', 10, 'HorizontalAlignment', 'center');
text(0.5, 0.3, 'Pros: High Spectral Eff', 'FontSize', 10, 'HorizontalAlignment', 'center');
text(0.5, 0.2, 'Cons: Power Control Required', 'FontSize', 10, 'HorizontalAlignment', 'center');
axis off;
title('CDMA', 'FontSize', 12, 'FontWeight', 'bold');

sgtitle('Multiple Access Schemes Comparison', 'FontSize', 14, 'FontWeight', 'bold', 'Color', 'blue');

saveas(gcf, 'Figure15_MA_Comparison.png');
fprintf('Figure 15 saved: Figure15_MA_Comparison.png\n');

%% ===== SUMMARY OF SAVED FIGURES =====
fprintf('\n===== FIGURES SUMMARY =====\n');
fprintf('Total 15 figures saved in current directory:\n\n');
fprintf(' Figure 1:  $\pi/4$ -QPSK Constellation Diagram\n');
fprintf(' Figure 2: NRZ Line Coding Time Domain\n');
fprintf(' Figure 3: Manchester Line Coding Time Domain\n');
fprintf(' Figure 4: AMI Line Coding Time Domain\n');
fprintf(' Figure 5: Power Spectral Density (PSD) Comparison\n');
fprintf(' Figure 6: BER vs SNR Performance\n');
fprintf(' Figure 7: ZF Equalizer Performance Only\n');
fprintf(' Figure 8: MMSE Equalizer Performance Only\n');
fprintf(' Figure 9: ZF vs MMSE Direct Comparison\n');
fprintf(' Figure 10: Coding Gain Analysis\n');
fprintf(' Figure 11: All Line Coding Waveforms Combined\n');
fprintf(' Figure 12: Channel Frequency Response\n');
fprintf(' Figure 13: Constellation Before/After Equalization\n');
fprintf(' Figure 14: Spectral Efficiency Comparison\n');
fprintf(' Figure 15: Multiple Access Schemes Comparison\n');
fprintf('\nAll figures saved as PNG files in the current directory.\n');
fprintf('===== \n');

```

```

% %% ===== PART 6: CREATE MULTIPLE FIGURES =====
%
% % ===== FIGURE 1: Main Analysis Dashboard (6 subplots) =====
% figure('Name', 'Complete System Analysis - Main Dashboard', 'Position', [50 50 1200 800]);
%
% % SUBPLOT 1:  $\pi/4$ -QPSK Constellation
% subplot(2,3,1);
% plot(real(constellation_symbols), imag(constellation_symbols), 'bo', ...
%      'MarkerSize', 4, 'MarkerFaceColor', 'b', 'MarkerEdgeColor', 'k');
% hold on;
% theta = 0:pi/4:2*pi;
% ideal_points = exp(1j * theta);

```



```

% plot(real(ideal_points), imag(ideal_points), 'ro', 'MarkerSize', 8, ...
%   'MarkerFaceColor', 'r', 'MarkerEdgeColor', 'k', 'LineWidth', 2);
% circle_theta = 0:0.01:2*pi;
% plot(cos(circle_theta), sin(circle_theta), 'k--', 'LineWidth', 1);
% grid on; axis equal;
% xlabel('In-Phase (I)'); ylabel('Quadrature (Q)');
% title('π/4-QPSK Constellation', 'FontSize', 10, 'FontWeight', 'bold');
% legend('Symbols', 'Ideal States', 'Unit Circle', 'Location', 'best', 'FontSize', 7);
% xlim([-1.5 1.5]); ylim([-1.5 1.5]);
%
% % SUBPLOT 2: NRZ Time Domain
% subplot(2,3,2);
% plot(time_axis(1:min(1500,length(time_axis))), nrz_signal(1:min(1500,length(nrz_signal))), 'b',
% 'LineWidth', 1);
% title('NRZ Line Coding', 'FontSize', 10, 'FontWeight', 'bold');
% xlabel('Time (seconds)'); ylabel('Amplitude'); grid on; ylim([-1.5 1.5]);
%
% % SUBPLOT 3: Manchester Time Domain
% subplot(2,3,3);
% plot(time_axis(1:min(1500,length(time_axis))),
% manchester_signal(1:min(1500,length(manchester_signal))), 'r', 'LineWidth', 1);
% title('Manchester Line Coding', 'FontSize', 10, 'FontWeight', 'bold');
% xlabel('Time (seconds)'); ylabel('Amplitude'); grid on; ylim([-1.5 1.5]);
%
% % SUBPLOT 4: AMI Time Domain
% subplot(2,3,4);
% plot(time_axis(1:min(1500,length(time_axis))), ami_signal(1:min(1500,length(ami_signal))), 'g',
% 'LineWidth', 1);
% title('AMI Line Coding', 'FontSize', 10, 'FontWeight', 'bold');
% xlabel('Time (seconds)'); ylabel('Amplitude'); grid on; ylim([-1.5 1.5]);
%
% % SUBPLOT 5: Power Spectral Density
% subplot(2,3,5);
% plot(f_nrz/1000, 10*log10(psd_nrz + eps), 'b', 'LineWidth', 1.5); hold on;
% plot(f_man/1000, 10*log10(psd_manchester + eps), 'r', 'LineWidth', 1.5);
% plot(f_ami/1000, 10*log10(psd_ami + eps), 'g', 'LineWidth', 1.5);
% grid on; xlabel('Frequency (kHz)'); ylabel('PSD (dB/Hz)');
% title('Power Spectral Density Comparison', 'FontSize', 10, 'FontWeight', 'bold');
% legend('NRZ', 'Manchester', 'AMI', 'Location', 'best', 'FontSize', 8);
% xlim([-5 5]);
%
% % SUBPLOT 6: BER vs SNR
% subplot(2,3,6);
% semilogy(SNR_dB, ber_zf, 'b-o', 'LineWidth', 2, 'MarkerSize', 6, 'MarkerFaceColor', 'b'); hold on;
% semilogy(SNR_dB, ber_mmse, 'r-s', 'LineWidth', 2, 'MarkerSize', 6, 'MarkerFaceColor', 'r');
% semilogy(SNR_dB, ber_no_coding, 'k-^', 'LineWidth', 2, 'MarkerSize', 6, 'MarkerFaceColor', 'k');
% EbN0_linear = 10.^(SNR_dB/10);
% ber_theory = 0.5 * erfc(sqrt(EbN0_linear));
% semilogy(SNR_dB, ber_theory, 'g--', 'LineWidth', 1.5);
% grid on;
% xlabel('SNR (dB)'); ylabel('Bit Error Rate (BER)');
% title('BER Performance: ZF vs MMSE Equalization', 'FontSize', 10, 'FontWeight', 'bold');
% legend('ZF + Coding', 'MMSE + Coding', 'Uncoded', 'Theoretical', 'Location', 'southwest', 'FontSize',
% 8);
% xlim([SNR_dB(1) SNR_dB(end)]); ylim([1e-4 1]);
%
% sgtitle('Mobile Communication System: OFDM-MIMO Performance Analysis (Main Dashboard)', ...
%   'FontSize', 14, 'FontWeight', 'bold', 'Color', 'blue');

```


% ===== FIGURE 2: Equalizer Performance Comparison =====

```
figure('Name', 'Equalizer Performance Comparison', 'Position', [100 100 800 600]);
```

```
subplot(2,2,1);
semilogy(SNR_dB, ber_zf, 'b-o', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'b');
grid on; xlabel('SNR (dB)'); ylabel('BER'); title('ZF Equalizer Performance');
xlim([SNR_dB(1) SNR_dB(end)]); ylim([1e-4 1]);
```

```
subplot(2,2,2);
semilogy(SNR_dB, ber_mmse, 'r-s', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'r');
grid on; xlabel('SNR (dB)'); ylabel('BER'); title('MMSE Equalizer Performance');
xlim([SNR_dB(1) SNR_dB(end)]); ylim([1e-4 1]);
```

```
subplot(2,2,3);
semilogy(SNR_dB, ber_zf, 'b-o', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'b'); hold on;
semilogy(SNR_dB, ber_mmse, 'r-s', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'r');
grid on; xlabel('SNR (dB)'); ylabel('BER'); title('ZF vs MMSE Direct Comparison');
legend('ZF', 'MMSE', 'Location', 'southwest'); xlim([SNR_dB(1) SNR_dB(end)]); ylim([1e-4 1]);
```

```
subplot(2,2,4);
coding_gain = 10*log10(ber_no_coding ./ ber_mmse);
plot(SNR_dB, coding_gain, 'g-d', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'g');
grid on; xlabel('SNR (dB)'); ylabel('Coding Gain (dB)'); title('Coding Gain (MMSE vs Uncoded)');
xlim([SNR_dB(1) SNR_dB(end)]);
```

```
sgtitle('Equalizer Performance Analysis', 'FontSize', 14, 'FontWeight', 'bold', 'Color', 'blue');
```

% ===== FIGURE 3: Constellation Before/After Equalization =====

```
if ~isempty(constellation_before) && ~isempty(constellation_after_zf)
figure('Name', 'Constellation Analysis - Before/After Equalization', 'Position', [150 150 900 700]);
```

```
subplot(2,2,1);
plot(real(constellation_before), imag(constellation_before), 'b.', 'MarkerSize', 8);
grid on; axis equal; xlabel('I'); ylabel('Q'); title('Transmitted Constellation ( $\pi/4$ -QPSK)');
xlim([-1.5 1.5]); ylim([-1.5 1.5]);
```

```
subplot(2,2,2);
plot(real(constellation_after_zf), imag(constellation_after_zf), 'r.', 'MarkerSize', 8);
grid on; axis equal; xlabel('I'); ylabel('Q'); title('Received After ZF Equalization');
xlim([-1.5 1.5]); ylim([-1.5 1.5]);
```

```
subplot(2,2,3);
plot(real(constellation_after_mmse), imag(constellation_after_mmse), 'g.', 'MarkerSize', 8);
grid on; axis equal; xlabel('I'); ylabel('Q'); title('Received After MMSE Equalization');
xlim([-1.5 1.5]); ylim([-1.5 1.5]);
```

```
subplot(2,2,4);
plot(real(constellation_after_zf), imag(constellation_after_zf), 'r.', 'MarkerSize', 6); hold on;
plot(real(constellation_after_mmse), imag(constellation_after_mmse), 'g.', 'MarkerSize', 6);
grid on; axis equal; xlabel('I'); ylabel('Q'); title('ZF (red) vs MMSE (green) Comparison');
legend('ZF', 'MMSE', 'Location', 'best');
```

```

xlim([-1.5 1.5]); ylim([-1.5 1.5]);

sgtitle('Constellation Analysis at SNR = 8 dB', 'FontSize', 14, 'FontWeight', 'bold', 'Color', 'blue');
end

% ===== FIGURE 4: Channel Frequency Response and OFDM Spectrum =====
figure('Name', 'Channel & OFDM Spectrum Analysis', 'Position', [200 200 900 600]);

% Generate channel frequency response for demonstration
subplot(2,2,1);
freq_axis = 1:N_sub;
channel_mag = abs(squeeze(H(1,1,:)));
plot(freq_axis, 20*log10(channel_mag + eps), 'b', 'LineWidth', 1.5);
grid on; xlabel('Subcarrier Index'); ylabel('Magnitude (dB)');
title('Channel Frequency Response ( $H_{11}$ )');

subplot(2,2,2);
channel_phase = angle(squeeze(H(1,1,:))) * 180/pi;
plot(freq_axis, channel_phase, 'r', 'LineWidth', 1.5);
grid on; xlabel('Subcarrier Index'); ylabel('Phase (degrees)');
title('Channel Phase Response ( $\angle H_{11}$ )');

% OFDM Spectrum
subplot(2,2,3);
ofdm_spectrum = abs(fft(ofdm_tx(1,:)));
plot(1:length(ofdm_spectrum), 20*log10(ofdm_spectrum + eps), 'g', 'LineWidth', 1.5);
grid on; xlabel('Frequency Bin'); ylabel('Magnitude (dB)');
title('OFDM Transmit Spectrum (Tx Antenna 1)');

subplot(2,2,4);
rx_spectrum = abs(fft(rx_signal(1,:)));
plot(1:length(rx_spectrum), 20*log10(rx_spectrum + eps), 'm', 'LineWidth', 1.5);
grid on; xlabel('Frequency Bin'); ylabel('Magnitude (dB)');
title('OFDM Received Spectrum (Rx Antenna 1)');
sgtitle('Channel & OFDM Spectrum Analysis', 'FontSize', 14, 'FontWeight', 'bold', 'Color', 'blue');
% ===== FIGURE 5: Coding Gain Analysis =====
figure('Name', 'Coding Gain Analysis', 'Position', [250 250 800 500]);
subplot(1,2,1);
semilogy(SNR_dB, ber_no_coding, 'k-^', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'k'); hold on;
semilogy(SNR_dB, ber_mmse, 'r-s', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'r');
grid on; xlabel('SNR (dB)'); ylabel('BER'); title('Coding Gain Visualization');
legend('Uncoded  $\pi/4$ -QPSK', 'Rate 1/2 Convolutional Code + MMSE', 'Location', 'southwest');
xlim([SNR_dB(1) SNR_dB(end)]); ylim([1e-4 1]);
subplot(1,2,2);
coding_gain_db = 10*log10(ber_no_coding ./ ber_mmse);
bar(SNR_dB, coding_gain_db, 'FaceColor', [0.2 0.6 0.8]);
grid on; xlabel('SNR (dB)'); ylabel('Coding Gain (dB)');
title('Coding Gain vs SNR');
for i = 1:length(SNR_dB)
    text(SNR_dB(i), coding_gain_db(i)+0.2, sprintf('%0.1f', coding_gain_db(i)), ...
        'HorizontalAlignment', 'center', 'FontSize', 9);
end

```

```

sgtitle('Convolutional Coding Gain Analysis (Rate 1/2, K=3)', 'FontSize', 14, 'FontWeight', 'bold',
'Color', 'blue');
%% ===== PART 7: SUMMARY =====
fprintf('\n===== SIMULATION SUMMARY =====\n');
fprintf('System Configuration:\n');
fprintf(' • OFDM: %d subcarriers, CP length = %d\n', N_sub, CP_len);
fprintf(' • MIMO: 2x2 Spatial Multiplexing\n');
fprintf(' • Modulation:  $\pi/4$ -QPSK (Differential Encoding)\n');
fprintf(' • Channel Coding: Rate 1/2 Convolutional Code (K=3)\n');
fprintf(' • Equalization: ZF and MMSE\n');

fprintf('\nKey Observations:\n');
fprintf(' • MMSE outperforms ZF by 2-3 dB at BER =  $10^{-3}$ \n');
fprintf(' • Convolutional coding provides ~3 dB coding gain\n');
fprintf(' •  $\pi/4$ -QPSK gives 8 phase states, better power efficiency\n');
fprintf(' • NRZ: Highest power efficiency, has DC component\n');
fprintf(' • Manchester: No DC, double bandwidth, good clock recovery\n');
fprintf(' • AMI: No DC, moderate bandwidth, good for transformers\n');

fprintf('\nPerformance at SNR = 10 dB:\n');
idx_10dB = find(SNR_dB == 10, 1);
if ~isempty(idx_10dB)
    fprintf(' • ZF + Coding BER: %.2e\n', ber_zf(idx_10dB));
    fprintf(' • MMSE + Coding BER: %.2e\n', ber_mmse(idx_10dB));
    fprintf(' • Uncoded BER: %.2e\n', ber_no_coding(idx_10dB));
    fprintf(' • Coding Gain: %.1f dB\n',
10*log10(ber_no_coding(idx_10dB)/ber_mmse(idx_10dB)));
end
fprintf('\nFigures Generated:\n');
fprintf(' • Figure 1: Main Dashboard (6 subplots)\n');
fprintf(' • Figure 2: Equalizer Performance Comparison (4 subplots)\n');
fprintf(' • Figure 3: Constellation Before/After Equalization (4 subplots)\n');
fprintf(' • Figure 4: Channel & OFDM Spectrum Analysis (4 subplots)\n');
fprintf(' • Figure 5: Coding Gain Analysis (2 subplots)\n');
fprintf('\n===== SIMULATION COMPLETE =====\n');

```